

1-Introduction



Sistemas Operativos

Introduction 1

1

Topics

- OS Definition
- OS Functions
- OS Organization
- OS Operation
- Typical OS Structure



Sistemas Operativos

Introduction 2

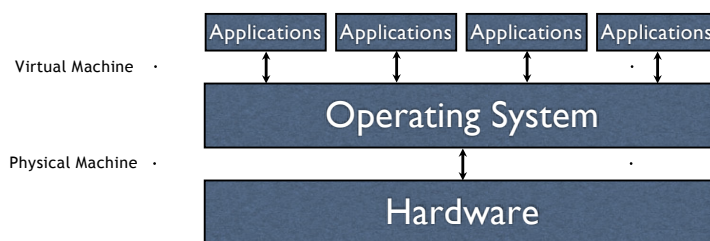
2

What is an OS?

- It is a program that allows the user to run other programs.
 - But different from a command interpreter or window manager...
- It is a program that allows controlled and efficient access to computer resources
 - CPU (process manager), memory (memory manager), monitor, keyboard, mouse (peripheral manager), persistent storage (file manager) and the network.



OS Functions



- Create a virtual machine on top of the physical machine that provides the basic logical resources needed for application development.
- Independent of the hardware it runs on.



OS Functions

- **Supervisor**
 - Controls program execution to prevent errors and improper use of the computer
- **Resource Manager**
 - It would be difficult for applications to control all aspects of the physical machine (interrupts, memory organization, devices, ...)
 - The OS allows abstracting physical resources, offering applications a set of logical resources



OS Functions

Logical Resources	Virtualized Physical Resources
Processes	CPU
Virtual Address Spaces	RAM Memory, Memory Management Unit
Files	Disks and mass storage devices
Virtual Devices	Physical Devices
Communication Channels	Shared memory and data networks
Users	Human Users

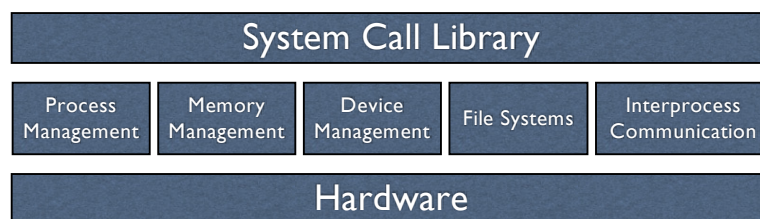


OS Functions

- Provide an access/management interface for logical resources
- There are two interfaces:
 1. Operational Interface
 - Commands to use / create / delete resources
 2. OS System Call Library
 - API to interact with OS resources



OS Organization



- The OS is divided into:
 - Kernel (*kernel*) - divided into modules
 - System call library (*system calls*)
 - Processes from the Operating System



Kernel Functions

- Controls process execution: allowing their creation, termination and inter-process communication.
- Schedules processes fairly for execution on the processor or processors
- Reserves memory for a running process. If the system runs low on free memory, it frees memory by writing memory regions to secondary storage.
- Allocates secondary memory to store and retrieve data (file systems), ensuring appropriate access permissions and mutual exclusion.
- Allows processes to control access to peripheral devices: monitors, keyboard, mouse, disk drives, networks - also enforcing appropriate access permissions and mutual exclusion.



Kernel Mode vs. User Mode

- The processor contains certain instructions only available to “programas privilegiados” como o SO.
 - change memory mappings, halt the processor, modify timers, define interrupt vectors, access restricted memory...
- They must also be forbidden to user applications to ensure system integrity.
- The processor has a “flag” that indicates whether it is running in:
 - **Kernel Mode** : with full access to the system
 - **User Mode**: without access to restricted operations



User Mode → Kernel Mode

- **Software Interrupts (“traps”)**
 - A user-programmed interrupt that forces the program to jump to a well-known address based on the interrupt number.
 - It is the way to invoke a system call.
 - Occurs in the context of the running process.
- **Exceptions (access violations)**
 - Interrupt generated by the processor in response to an access violation or problem executing an instruction.
 - Also occurs in the context of the running process.
- **Hardware Interrupts**
 - Occur when an event in an external system (peripheral) generates an interrupt signal.
 - They are unpredictable! They can occur at any time...
 - do not depend on the instruction being executed
 - May force a context switch



System Calls

- **Access to privileged functions requires OS intervention**
 - Examples: creating or reading a file, creating a new process,...
 - The way to do this is to invoke a **System Call**.
- **How to invoke a system call?**
 - Place the required arguments and the system call number on the stack.
 - Invoke the OS using a software interrupt.
- **Executing a system call**
 1. The kernel verifies the legality of the requested operation,
 2. Preserves all process registers,
 3. Executes the operation copying the results to the stack,
 4. Restores the registers and returns from the interrupt



Typical OS Structure

